

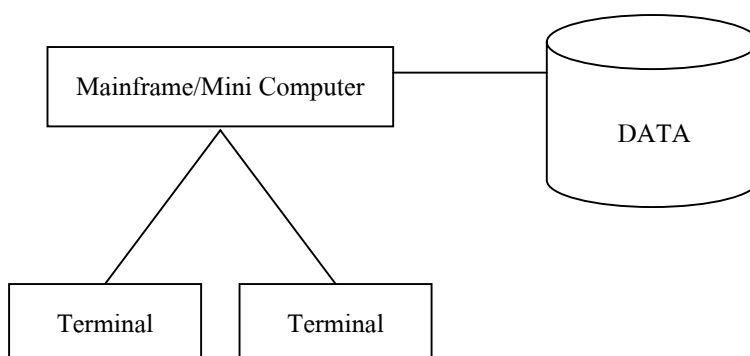
บทที่ 3

วิวัฒนาการของสถาปัตยกรรมแบบกระจาย

3.1 สถาปัตยกรรมแบบหนึ่งเทียร์ (One Tier Architecture/Centralize Host)

สถาปัตยกรรมแบบหนึ่งเทียร์ คือระบบผู้ใช้หลายคนที่เรียบง่ายที่สุด โดยจะมีการเก็บข้อมูลและประมวลผลอยู่ที่คอมพิวเตอร์ส่วนกลางทั้งหมด (เช่น เมนเฟรมคอมพิวเตอร์หรือมินิคอมพิวเตอร์) ผู้ใช้จะติดต่อกับคอมพิวเตอร์ส่วนกลางโดยผ่านทางเทอร์มินอล ซึ่งเป็นเพียงแค่อุปกรณ์อินพุตและเอาต์พุตเพียงอย่างเดียว ไม่มีการประมวลผลในระบบเช่นนี้มีข้อเสียคือ

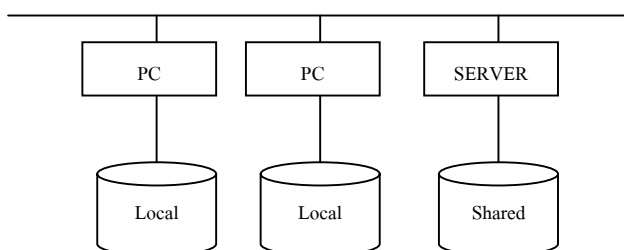
- ❑ การที่ทุกๆ กระบวนการทำงานทำในคอมพิวเตอร์ส่วนกลางทั้งหมด ทำให้ต้องแบ่งกำลังการประมวลผลไปให้ผู้ใช้แต่ละคน อาจมีผลกระทบต่อเวลาการตอบสนอง ทำให้ตอบสนองได้ช้าลง
- ❑ ถ้าเกิดความขัดข้องที่คอมพิวเตอร์ส่วนกลาง ผู้ใช้ทุกคนจะไม่สามารถใช้งานได้
- ❑ ค่าใช้จ่ายในการบำรุงรักษาทางด้านฮาร์ดแวร์ และซอฟต์แวร์สูงมาก



รูปที่ 3-1 สถาปัตยกรรมแบบหนึ่งเทียร์

3.2 เครือข่ายภายในและการแบ่งปันทรัพยากรภายในเครือข่าย (PC LANs/ทรัพยากร Sharing LANs)

คอมพิวเตอร์ส่วนบุคคล (Personal Computer: PC) ได้ถือกำเนิดขึ้นมาเพื่อทดแทนความไม่ยืดหยุ่นของระบบเมนเฟรม พิสูจน์ว่ามีค่าใช้จ่ายทางด้านซอฟต์แวร์และฮาร์ดแวร์ถูกกว่าระบบเมนเฟรมมาก พิธีในยุคแรกเริ่มเป็น พิธีที่มีการใช้งานอยู่เครื่องเดียว ในก้าวต่อมาจึงมีการเชื่อมต่อพีซีเข้าด้วยกันผ่านเครือข่ายภายใน (Local Area Network: LAN)



รูปที่ 3-2 เครือข่ายภายในของเครื่องคอมพิวเตอร์ส่วนบุคคล

ในส่วนที่เป็นเซิร์ฟเวอร์นั้น อาจเป็นเซิร์ฟเวอร์ที่ให้บริการเพิ่มข้อมูล หรือเซิร์ฟเวอร์ที่ให้บริการเครื่องพิมพ์ (File/Print Server) ก็ได้ แต่แอปพลิเคชันจะยังคงทำงานอยู่บนพีซีของตัวเอง การที่ข้อมูลอยู่บนคิสก์ที่ใช้ร่วมกันที่เซิร์ฟเวอร์ และแอปพลิเคชันทำงานอยู่ที่พีซี ทำให้เกิดการถ่ายโอนข้อมูลจำนวนมากและเป็นไปอย่างต่อเนื่อง ยกตัวอย่างเช่น การคิวรีข้อมูลจากข้อมูลทั้งหมด 10,000 เรคคอร์ด เซิร์ฟเวอร์จะต้องส่งข้อมูลทั้ง 10,000 เรคคอร์ดนั้นมาประมวลผลคำสั่ง SQL ที่เครื่องไคลเอนต์

3.3 สถาปัตยกรรมแบบสองเทียร์ (ไคลเอนต์/เซิร์ฟเวอร์)

3.3.1 ดาต้าเบสเซิร์ฟเวอร์ (Database Server)

ในระบบดาต้าเบสเซิร์ฟเวอร์นั้นดาต้าและดาต้าเบสเอ็นจิน (DBMS) จะรันอยู่ที่เซิร์ฟเวอร์ ส่วนหน้าจอใช้งานจะรันอยู่บนพีซี ซึ่งเรียกว่าไคลเอนต์ ในการสืบค้นข้อมูลนั้น จะส่งคำสั่ง SQL statement ไปสืบค้นที่เซิร์ฟเวอร์และเซิร์ฟเวอร์จะส่งเฉพาะผลลัพธ์กลับไปไคลเอนต์ ซึ่งจะช่วยลดความหนาแน่นของข้อมูลในระบบเครือข่ายลงได้ ไคลเอนต์กับเซิร์ฟเวอร์จะเป็นอิสระต่อกัน แต่จะมีแอปพลิเคชันทำงานร่วมกันอยู่ แอปพลิเคชันฝั่งเซิร์ฟเวอร์คือ ดาต้าเบสเอ็นจิน ส่วนแอปพลิเคชันฝั่งไคลเอนต์ ก็คือแอปพลิเคชันต่างๆ ที่ต้องการติดต่อกับดาต้าเบส แอปพลิเคชันทั้งสองส่วนจะติดต่อสื่อสารกันผ่านเน็ตเวิร์กโพรโตคอล ซึ่งเป็นความรับผิดชอบของ DBMS โดย DBMS จะจัดเตรียมโครงสร้างพื้นฐานทั้งหมดให้

ข้อเสียของไคลเอนต์/เซิร์ฟเวอร์ ก็คือการที่แอปพลิเคชันนั้นรันอยู่บนไคลเอนต์ การแก้ไขแอปพลิเคชันจึงต้องทำทุกๆ ไคลเอนต์ ทำให้มีค่าใช้จ่ายในการบำรุงรักษาสูงมาก

3.3.2 แอปพลิเคชันเซิร์ฟเวอร์

แอปพลิเคชันเซิร์ฟเวอร์จะมีความสามารถมากกว่าการแชร์ไฟล์หรือเครื่องพิมพ์ในเครือข่ายภายในหรือการจัดการดาต้าเบสในดาต้าเบสเซิร์ฟเวอร์ แอปพลิเคชันเซิร์ฟเวอร์นั้นสามารถใส่แอปพลิเคชันที่มีฟังก์ชันการทำงานต่างๆ ไว้ และเรียกใช้เมื่อต้องการได้ โดยแอปพลิเคชันที่จะเรียกใช้ฟังก์ชันที่อยู่บนเซิร์ฟเวอร์จะต้องใช้เน็ตเวิร์กโพรโตคอล เช่น TCP/IP, SPX/IPX หรือโพรโตคอลในระดับสูง เช่น DCOM, COBRA, RMI

3.4 สถาปัตยกรรมแบบสามเทียร์

ในระบบสถาปัตยกรรมแบบสองเทียร์นั้น ทุกๆ ไคลเอนต์ติดต่อโดยตรงกับดาต้าเบส ซึ่งการทำเช่นนี้เหมาะสมกับงานขนาดเล็ก แต่ไม่เหมาะสมกับองค์กรใหญ่ๆ ที่มีผู้ใช้จำนวนมาก โดยเฉพาะอย่างยิ่งในระบบอินเตอร์เน็ตจะมีผู้ใช้ที่ใช้งานพร้อมกันเป็นจำนวนมากในช่วงเวลาที่มีการใช้งานมากๆ (peak time) จะทำให้เกิดปัญหาการขาดแคลนทรัพยากรได้ ด้วยเหตุนี้ เทียร์ที่สามคือแอปพลิเคชันเทียร์หรือบิสิเนสเทียร์ จึงเกิดขึ้นมาคั่นระหว่างพรีเซนเตชันเทียร์และดาต้าเบสเทียร์เพื่อเพิ่มความยืดหยุ่น และมีความสามารถในการกระจายโหลดได้มากขึ้น ยกตัวอย่างเช่นบิสิเนสเลเยอร์สามารถจัดการพูล (Pool) การเชื่อมต่อของดาต้าเบสได้ โดยจะไม่ยกเลิกการเชื่อมต่อกับดาต้าเบสเมื่อไม่ใช้งาน แต่จะเก็บลงพูลแทน ทำให้

รองรับไคลเอนต์ได้มากขึ้น บิสิเนสเลเยอร์นั้นจะแยกไคลเอนต์ออกจากดาต้าเบส ไม่ให้ขึ้นต่อกันโดยตรง ดังนั้นดาต้าเบสสามารถเปลี่ยนแปลงได้ โดยไม่มีผลกระทบต่อโค้ดในพีริเซนเตชันเลเยอร์เลย บิสิเนสเลเยอร์ก็สามารถสร้าง Business Rule ที่เป็นสามัญซึ่งจะใช้ได้ในหลายๆ ไคลเอนต์แอปพลิเคชันทำให้เกิดการนำโค้ดต้นฉบับมาใช้ใหม่ ถ้าฟังก์ชันทางธุรกิจเปลี่ยนแปลงไป ก็ปรับปรุงเฉพาะบิสิเนสเลเยอร์เท่านั้น

3.4.1 วินโดวส์ดีเอ็นเอ (Windows DNA)

ไมโครซอฟต์สนับสนุนการกระจายแอปพลิเคชันด้วยสถาปัตยกรรมวินโดวส์ดีเอ็นเอ (Windows Distributed Internet Application Architecture: Windows DNA)

วินโดวส์ดีเอ็นเอนำเสนอแอปพลิเคชันแบบสาม-tier โดยอยู่บนพื้นฐานของ Component Object Model (COM) จุดประสงค์ของวินโดวส์ดีเอ็นเอ คือ การแยกโลจิกทางธุรกิจออกจากระบบไคลเอนต์/เซิร์ฟเวอร์ไปสู่วินโดวส์ทีเยอร์ซึ่งรันอยู่บนวินโดวส์ 2000

วินโดวส์ดีเอ็นเอไม่ใช่ผลิตภัณฑ์ของบริษัทใดๆ แต่เป็นแนวทางการพัฒนาแอปพลิเคชันแบบสาม-tier โดยทางไมโครซอฟต์ได้จัดเตรียมแพลตฟอร์ม คอมโพเนนต์เทคโนโลยี โครงสร้างภายในแอปพลิเคชัน บริการและเครื่องมือต่างๆ ให้ หัวใจของวินโดวส์ดีเอ็นเอ นั้น คือการรวมโมเดลการเขียนโปรแกรมที่อยู่บนพื้นฐานของ COM เข้าด้วยกัน

3.4.1.1 พีริเซนเตชันทีเยอร์

Rich Client เป็น Win 32 แอปพลิเคชันซึ่งสามารถเข้าถึงทุก ๆ ทรัพยากรของคอมพิวเตอร์ที่เป็นไคลเอนต์ได้ Rich Client ติดต่อกับ Middle Tier โดยผ่านทาง DCOM

Thin Client จะติดต่อกับ Web Server ผ่านทาง HTTP Protocol เทคโนโลยีของไมโครซอฟต์สำหรับ Thin Client ก็คือ Web Browser (Internet Explorer)

3.4.1.2 บิสิเนสโลจิกทีเยอร์ (Business Logic Tier)

บิสิเนสโลจิกเลเยอร์หรือ Middle Tier นั้น เป็นส่วนที่สำคัญที่สุดของสถาปัตยกรรมแบบสาม-tier บริการใหม่ ๆ ของไมโครซอฟต์จะอยู่ที่เลเยอร์นี้ เช่น

- Web Service โดยผ่านทาง Internet Information Server (IIS)
- Transaction และ Component Service ผ่านทาง Microsoft Transaction Server (MTS)
- Queuing และ Asynchronous Service ผ่านทาง Microsoft Message Queue (MSMQ)
- Server Side Scripting ผ่านทาง Active Server Page (ASP) ซึ่งอยู่บน IIS

3.4.1.3 ดาต้าแอ็กเซสเลเยอร์ (Data Access Layer)

ในดาต้าแอ็กเซสเลเยอร์นี้ ไมโครซอฟต์มีผลิตภัณฑ์ที่เป็นตัวจัดการฐานข้อมูลคือ Microsoft SQL Server 2000 แต่ก็ยังกำหนด Data Access Interface ซึ่งสนับสนุนผลิตภัณฑ์อื่น ๆ ด้วย เช่น Open Database Connectivity (ODBC), OLE DB

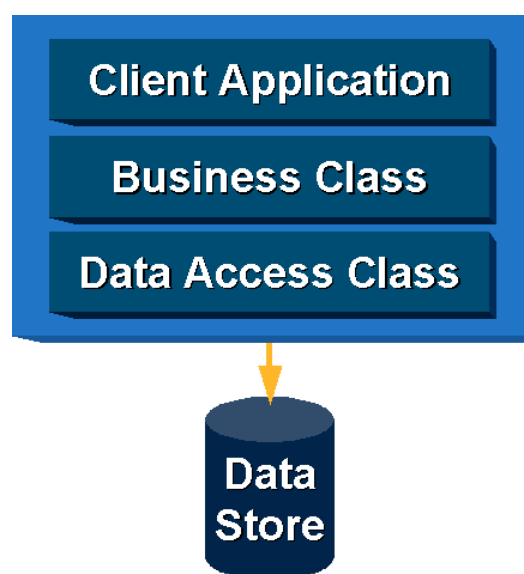
คุณสมบัติที่สำคัญที่สุดของ Data Access Technology ของไมโครซอฟต์ ก็คือ ความสามารถในการเข้าถึงทุก ๆ แหล่งข้อมูลได้อย่างสะดวกโดยผ่าน Interface เดียวกัน ซึ่งเรียกว่า Uniform Data Access (UDA)

3.5 สถาปัตยกรรมแบบหลายเทียร์(N-Tier)

3.5.1 แอปพลิเคชันแบบหลายเทียร์ทางตรรก

วิธีที่ดีที่สุดในการพัฒนาแอปพลิเคชันโดยใช้ .NET นั้น คือการแบ่งลอจิกคอลลอกเป็นคลาสที่ไม่ขึ้นต่อกัน โดยทั่วไปแอปพลิเคชันในทางธุรกิจ

จะเกี่ยวข้องกับกฎทางธุรกิจ ส่วนประกอบของชั้นข้อมูล และโค้ดที่ใช้ส่วนประกอบนั้นๆ



รูปที่ 3-3 สถาปัตยกรรมแบบ เอ็น เทียร์

สำหรับเทคนิคนี้ การใช้ลอจิกคอลล n-tier ในการพัฒนาแอปพลิเคชัน สามารถใช้ได้ทั้งในแอปพลิเคชันขนาดเล็ก กลาง และใหญ่ ร่วมกับเดสก์ท็อปและเว็บแอปพลิเคชัน

ทางเลือกโดยทั่วไปในการพัฒนาแอปพลิเคชัน

- สร้างยูสเซอร์อินเทอร์เฟซ (User Interface) ทั้งวินโดวส์ฟอร์ม หรือ เว็บฟอร์ม
- สร้างกฎทางธุรกิจโดยแยกออกเป็นคลาสไลบรารี
- สร้างชั้นข้อมูลโดยแยกออกเป็นคลาสไลบรารี ชั้นข้อมูลนี้จะใช้คลาสในการติดต่อไปยังตารางแต่ละตาราง เพื่อความยืดหยุ่นของคลาสดาต้าเซต

ข้อดี

- การรวมกฎทางธุรกิจเป็นคอมโพเนนต์จะทำให้ง่ายต่อการสร้าง การนำไปใช้ และการนำกลับมาใช้ใหม่ ทำให้ง่ายต่อการพัฒนาและการดูแล
- สามารถใช้ภาษาระดับสูง ในการพัฒนากฎทางธุรกิจ ซึ่งตรงข้ามกับการใช้สโตรโปรซีเจอร์ (Stored procedure) และการตรวจสอบการ

จำกัดภาษา SQL

- รวมการเข้าถึงข้อมูลเป็นคอมโพเนนต์ ทำให้ลดการเขียนโค้ดซ้ำๆกันในแอปพลิเคชัน แต่ละฟอร์มที่ต้องการติดต่อกับฐานข้อมูลจะใช้คอมโพเนนต์

ตัวเดียวกัน

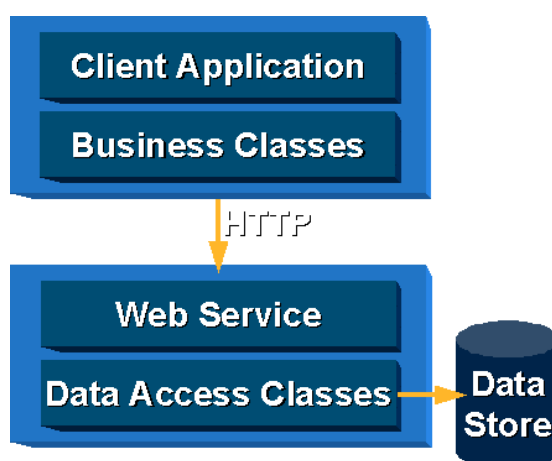
- หากใช้ดาต้าเซต(DataSet) จะทำให้สะดวกต่อการค้นหาคอลัมน์โดยใช้ อินเทลลิเซนซ์ (IntelliSense) แทนที่จะต้องจำ
- รวบรวม (Routines) ในการเข้าถึงข้อมูลจะช่วยให้การดูแล เนื่องจากการเปลี่ยนแปลงข้อมูลในการเข้าถึงข้อมูลจะต้องทำเพียงครั้งเดียว
- มีความยืดหยุ่นในการแบ่งสรรคอมโพเนนต์ให้กับเครื่องที่ทำงานเฉพาะอย่างได้ไม่ว่าเวลาใดก็ตาม ช่วยให้การขยายระบบให้รองรับการใช้งานได้มากขึ้น และง่ายต่อการรวบรวมโค้ด

ข้อเสีย

- การพัฒนายาวนานเนื่องจากต้องเสียเวลาในการแบ่งคอมโพเนนต์
- มีคอมโพเนนต์ที่ต้องติดตาม ทำให้การพัฒนาเมธอด (method) มีความยุ่งยากมากขึ้นเล็กน้อยที่จะทำให้ นักพัฒนาใหม่จะเข้าใจ

3.5.2 แอปพลิเคชัน N-Tier โดยใช้ XML เว็บเซอร์วิส

สถาปัตยกรรมนี้พัฒนาการออกแบบบล็อกคอล n-tier แอปพลิเคชันขึ้น โดยแบ่งออกเป็นหลายๆ เครื่อง XML เว็บเซอร์วิสที่ติดต่อไปยังชั้นข้อมูล



รูปที่ 3-4 สถาปัตยกรรมแบบ เอ็น เทียร์ ผ่านเว็บเซอร์วิส

โดยใช้ดาต้าเซตจะส่งข้อมูลกลับมาจากชั้น HTTP มายังชั้นของธุรกิจ แอปพลิเคชันทางฝั่งไคลเอนท์จะรับข้อมูลมาเพื่อใช้ในการแสดงผล

สำหรับเทคนิคนี้ การออกแบบเว็บเซอร์วิสโดยใช้ XML นั้นจะทำกับวินโดวส์ฟอร์มแอปพลิเคชัน แต่ผู้ใช้งานจะต้องติดต่อไปยังเครื่องที่ต้องการ และรับข้อมูลผ่านทางชั้น HTTP การเก็บกฎทางธุรกิจบนฝั่งไคลเอนท์นั้น จะทำให้เกิดปัญหาแพร่ฟิบบนเน็ตเวิร์ก แต่จะทำให้การดูแลลำบากขึ้นเมื่อมีการเปลี่ยนแปลงกฎบ่อยครั้ง

การทำงานในลักษณะนี้ จะใช้ได้กับเว็บเบสแอปพลิเคชันได้เช่นกัน โดยที่ข้อมูลจะต้องถูกเตรียมโดยเว็บเซิร์ฟเวอร์ และแสดงผลบนเว็บแอปพลิเคชันโดยเว็บเซิร์ฟเวอร์อีกเครื่องหนึ่ง ทางเลือกทั่วไปในการพัฒนา

- สร้างยูสเซอร์อินเทอร์เฟซ (User Interface) ทั้งวินโดวส์ฟอร์ม หรือ เว็บฟอร์ม
- สร้างกฎทางธุรกิจโดยแยกออกเป็นคลาสไลบรารี
- สร้างชั้นข้อมูลโดยแยกออกเป็นคลาสไลบรารี ชั้นข้อมูลนี้จะใช้คลาสในการติดต่อไปยังตาราง

แต่ละตาราง เพื่อความยืดหยุ่นของคลาสดาต้าเซต

ข้อดี

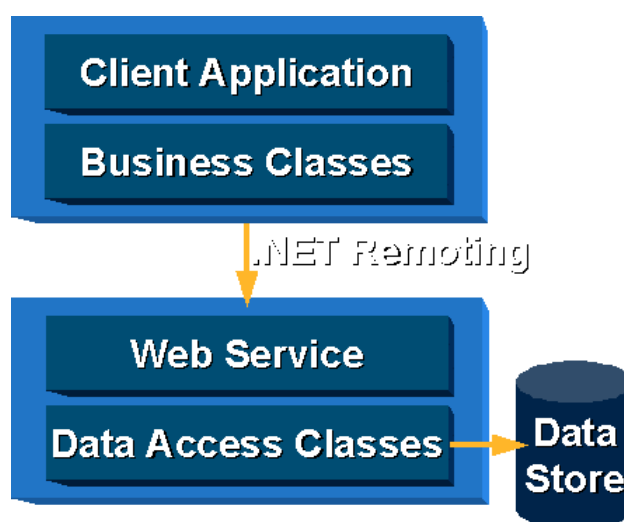
- การรวมกฎทางธุรกิจเป็นคอมโพเนนต์จะทำให้ง่ายต่อการสร้าง การนำไปใช้ และการนำกลับมาใช้ใหม่ ทำให้ง่ายต่อการพัฒนาและการดูแล
- สามารถใช้ภาษาระดับสูง ในการพัฒนากฎทางธุรกิจ ซึ่งตรงข้ามกับการใช้สโตรโปรซีเจอร์ (Stored procedure) และการตรวจสอบการจำกัดภาษา SQL

- รวมการเข้าถึงข้อมูลเป็นคอมโพเนนต์ ทำให้ลดการเขียนโค้ดซ้ำๆกันในแอปพลิเคชัน แต่
ละฟอร์มที่ต้องการติดต่อกับฐานข้อมูลจะใช้คอมโพเนนต์
ตัวเดียวกัน
- หากใช้ดาต้าเซต จะทำให้สะดวกต่อการค้นหาคอลัมน์โดยใช้ อินเทลลิเซนซ์ (IntelliSense)
แทนที่จะต้องจำ
- รวบรวม (Routines) ในการเข้าถึงข้อมูลจะช่วยในการดูแล เนื่องจากการเปลี่ยนแปลงข้อมูลใน
การเข้าถึงข้อมูลจะต้องทำเพียงครั้งเดียว
- มีความยืดหยุ่นในการแบ่งสรรคอมโพเนนต์ให้กับเครื่องที่ทำงานเฉพาะอย่างได้ไม่ว่าเวลาใดก็
ตาม ช่วยให้การขยายระบบให้รองรับการใช้งานได้มากขึ้น
และง่ายต่อการรวบรวมโค้ด
- ขั้นตอนการเข้าถึงข้อมูลเป็นศูนย์กลางในการเข้าถึงข้อมูล
- สามารถขยายระบบให้รองรับการใช้งานได้มากขึ้น โดยการเพิ่มเว็บฟอร์มเพื่อรองรับการร้องขอ
เพื่อเข้าถึงฐานข้อมูลของผู้ใช้
- ผู้ใช้สามารถติดต่อผ่านทางอินเทอร์เน็ตและยังคงเข้าถึงข้อมูลได้จากเดสก์ท็อปแอปพลิเคชัน
หรือเว็บเบสแอปพลิเคชัน

ข้อเสีย

- การพัฒนายาวนานเนื่องจากต้องเสียเวลาในการแบ่งคอมโพเนนต์
- มีคอมโพเนนต์ที่ต้องติดตาม ทำให้การพัฒนาเมธอด (method) มีความยุ่งยากมากขึ้นเล็กน้อยที่
จะทำให้ให้นักพัฒนาใหม่จะเข้าใจ

3.5.3 แอปพลิเคชัน N-Tier โดยใช้รีโมต .NET



รูปที่ 3-5 สถาปัตยกรรมแบบ เอ็น เทียร์ผ่าน .NET Remoting

การทำงาน ข้อดีและข้อเสียจะเหมือนกับ n-tier เว็บเซอร์วิส